Assignment 2: From Trees to Neural Networks

Comparing XGBoost and MLP on Home Credit Default Risk

Willow WU
UNI: yw4643

Code Repository: https://github.com/Cyriellewu/assignment2-ml

Abstract

This report compares two model families for binary classification on the Home Credit Default Risk dataset: Gradient Boosted Decision Trees (XGBoost) and Multi-Layer Perceptrons (MLP). The goal is to predict whether a loan applicant will default using a tabular dataset with mixed feature types, missing values, and strong class imbalance. I built separate preprocessing pipelines for tree-based and neural-network models, tuned key hyperparameters on a validation split, selected decision thresholds based on validation performance, visualized training behavior, and compared final test performance using Accuracy, Precision, Recall, F1-score, AUC-PR, threshold, and training time. Overall, XGBoost achieved the best ranking performance with a test AUC-PR of 0.250945 and the best F1-score of 0.311579, while MLP achieved slightly higher test accuracy but weaker ranking performance and much longer training time. These results suggest that XGBoost is the stronger and more practical choice for this structured credit-risk task.

1. Introduction

Credit default prediction is an important classification task in financial risk management. In this assignment, I compared two fundamentally different approaches: Gradient Boosted Decision Trees (GBDT), implemented with XGBoost, and Multi-Layer Perceptrons (MLP), implemented with `sklearn.neural_network.MLPClassifier`. The comparison is useful because the two model families learn in different ways: XGBoost builds an ensemble of decision trees stage by stage, while MLP learns nonlinear transformations through gradient-based optimization.

I selected the **Home Credit Default Risk** dataset. According to the experiment log, the raw dataset contains 307,511 rows and 122 columns. After feature engineering and removal of the target and ID field from the feature matrix, the processed feature table contains 307,511 rows and 125 columns. The target mean is 0.0807288, which indicates substantial class imbalance. The dataset was split into training, validation, and test sets with approximate proportions of 70%, 15%, and 15%, resulting in 215,257 training rows, 46,127 validation rows, and 46,127 test rows. The feature set contains 109 numeric variables and 16 categorical variables, and after one-hot encoding the transformed matrices contain 249 columns.

This report addresses four questions:

1. How should the dataset be prepared without introducing data leakage?
2. How do XGBoost hyperparameters affect validation performance and convergence?
3. How do MLP architecture and optimization settings affect validation performance?
4. Which model family is better suited for this tabular credit-default prediction problem?

2. Data Preparation

2.1 Preprocessing Strategy

The target variable is `TARGET`, and the customer ID variable `SK_ID_CURR` was excluded from the feature set. I also performed light feature engineering to improve the representation of key financial relationships:

- `DAYS_EMPLOYED_ANOM`: indicator for the anomalous value 365243
- `CREDIT_INCOME_RATIO`: `AMT_CREDIT/AMT_INCOME_TOTAL`
- `ANNUITY_INCOME_RATIO`: `AMT_ANNUITY/AMT_INCOME_TOTAL`
- `CREDIT_GOODS_RATIO`: `AMT_CREDIT/AMT_GOODS_PRICE`
- `AGE_YEARS`: derived from `DAYS_BIRTH`

Missing values were handled with `SimpleImputer`. Numeric features were imputed with the median. Categorical features were imputed with the most frequent category and then one-hot encoded. For XGBoost, I did not standardize features because tree-based models are insensitive to monotonic scaling. For MLP, numeric features were standardized with `StandardScaler`, since neural networks are more sensitive to feature scales and optimization stability.

2.2 Data Leakage Prevention

To avoid leakage, all preprocessors were fit on the **training split only**, and then applied to the validation and test sets. This is especially important for imputers, encoders, and scalers, since fitting them on the full dataset

would leak information from validation and test data into model development.

2.3 Dataset Summary

Table 1: Dataset and split summary

Item	Value
Raw data shape	307,511 × 122
Feature matrix shape	307,511 × 125
Target mean	0.080729
Train shape	215,257 × 125
Validation shape	46,127 × 125
Test shape	46,127 × 125
Numeric columns	109
Categorical columns	16
Encoded feature dimension	249
scale_pos_weight (XGBoost)	11.387466

3. XGBoost Experiments

3.1 Learning Rate Study

I first compared three learning rates: 0.01, 0.1, and 0.3. For each run, I trained an XGBoost classifier, evaluated on the validation split, and selected a classification threshold from the validation predictions. Table 2 reports the results.

Table 2: XGBoost validation performance for different learning rates

Model	Acc.	Prec.	Rec.	F1	AUC-PR	Thr.	LR	Time	Iter.
XGB_lr_0.01	0.8560	0.2500	0.3915	0.3051	0.2514	0.6592	0.01	66.34	999
XGB_lr_0.1	0.8570	0.2383	0.3510	0.2839	0.2311	0.6005	0.10	57.43	999
XGB_lr_0.3	0.8262	0.1962	0.3722	0.2569	0.1906	0.3620	0.30	60.81	998

The best validation AUC-PR, **0.251353**, was achieved by **learning rate = 0.01**. This setting also yielded the best validation F1-score among the three candidates. Although larger learning rates can sometimes accelerate optimization, here they reduced ranking quality substantially. Increasing the learning rate from 0.01 to 0.3 lowered AUC-PR from 0.251353 to 0.190631. The selected validation threshold for the best learning rate was 0.659177.

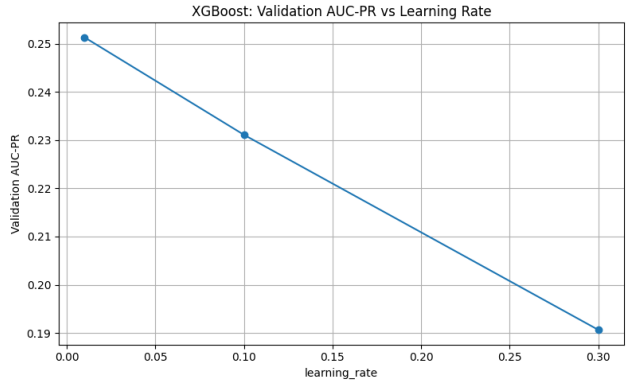
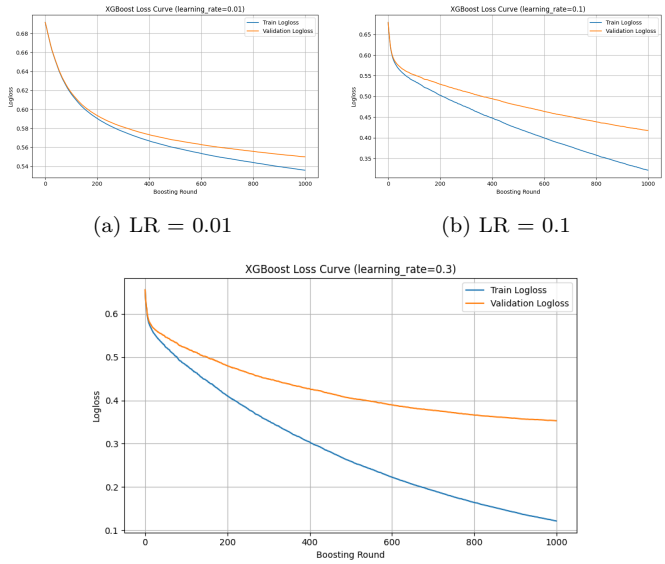


Figure 1: Validation AUC-PR under different XGBoost learning rates.



(a) LR = 0.01 (b) LR = 0.1

(c) LR = 0.3

Figure 2: Training and validation loss curves for XGBoost under different learning rates.

The loss curves show a consistent pattern. With learning rate 0.01, training and validation log loss decrease more smoothly and preserve the strongest validation ranking performance. With larger learning rates, training loss decreases rapidly, but validation quality deteriorates. This supports the choice of a smaller learning rate for the final XGBoost model.

3.2 Number of Estimators Study

After selecting the learning rate, I explicitly compared `n_estimators = 100`, 300, and 1000. The validation results are shown in Table 3.

Table 3: XGBoost validation performance for different numbers of estimators

Model	Acc.	Prec.	Rec.	F1	AUC-PR	Thr.	n_est.	Time
XGB_nest_1000	0.8560	0.2500	0.3915	0.3051	0.2514	0.6592	1000	49.48
XGB_nest_300	0.8371	0.2275	0.4251	0.2964	0.2388	0.6242	300	19.02
XGB_nest_100	0.8378	0.2214	0.4006	0.2852	0.2273	0.5803	100	7.91

The best validation AUC-PR was again achieved with **1000 estimators**. While smaller values reduced training time, they also lowered ranking performance. This indicates that, under the selected low learning rate, the model benefited from a larger number of boosting rounds.

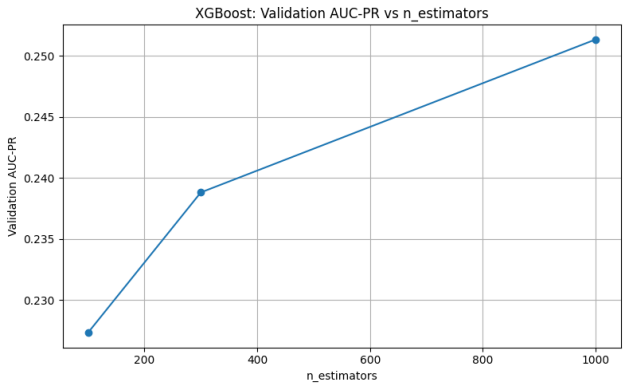


Figure 3: Validation AUC-PR for different values of `n_estimators`.

3.3 Hyperparameter Tuning

With learning rate fixed at 0.01 and `n_estimators = 1000`, I tuned `max_depth`, `subsample`, `reg_alpha`, and `reg_lambda`. The validation results are shown in Table 4.

Table 4: XGBoost validation performance under different parameter settings

Acc.	Prec.	Rec.	F1	AUC-PR	Thr.	depth	subs.	α	λ
0.8373	0.2327	0.4420	0.3048	0.2526	0.6327	6	0.6	0.0	1.0
0.8529	0.2477	0.4036	0.3070	0.2517	0.6543	6	0.8	1.0	2.0
0.8511	0.2448	0.4049	0.3051	0.2514	0.6517	6	0.8	0.5	1.0
0.8560	0.2500	0.3915	0.3051	0.2514	0.6592	6	0.8	0.0	1.0
0.8374	0.2323	0.4396	0.3039	0.2511	0.6325	6	1.0	0.0	1.0
0.8459	0.2397	0.4184	0.3048	0.2501	0.6526	4	0.8	0.0	1.0
0.8512	0.2436	0.4006	0.3029	0.2481	0.6256	8	0.8	0.0	1.0

The best validation AUC-PR, **0.252619**, was achieved with

$$\texttt{max_depth} = 6, \quad \texttt{subsample} = 0.6, \quad \texttt{reg_alpha} = 0.0, \quad \texttt{reg_lambda} = 1.0.$$

This setting also produced the highest validation recall among the candidate configurations. The results suggest that moderate depth and reduced subsampling helped the model generalize better. Increasing `max_depth` to 8 did not improve AUC-PR and increased training time substantially. Stronger regularization also failed to improve overall ranking performance. The selected validation threshold for the best configuration was **0.632711**.

3.4 Final XGBoost Test Performance

Using the best validation configuration, the final XGBoost model was retrained on train + validation data and evaluated on the test set. The final results are shown in Table 5.

Table 5: Final XGBoost test performance

Model	Acc.	Prec.	Rec.	F1	AUC-PR	0
Final_XGBoost	0.8370	0.2364	0.4570	0.3116	0.2509	

Compared with the earlier baseline version, this threshold-tuned XGBoost model behaves very differently: its recall on the positive class rises to 0.457035, and its F1-score rises to 0.311579. Accuracy drops to 0.836950, but this is expected because the selected threshold was optimized for better minority-class retrieval rather than majority-class accuracy. For this imbalanced task, the gain in recall and F1 is more meaningful than maximizing accuracy alone.

Table 6: Final XGBoost classification report on the test set

Class	Precision	Recall	F1-score	Support
0	0.95	0.87	0.91	42,403
1	0.24	0.46	0.31	3,724
Accuracy		0.84		46,127
Macro Avg	0.59	0.66	0.61	46,127
Weighted Avg	0.89	0.84	0.86	46,127

3.5 Feature Importance

The most important XGBoost features are listed in Table 7. The `EXT_SOURCE` variables remain dominant, followed by education, income type, gender, and several engineered financial features. This is consistent with the idea that external credit scores and applicant socioeconomic characteristics are highly informative for default prediction.

Table 7: Top 10 XGBoost feature importances

Rank	Feature	Importance
1	num__EXT_SOURCE_3	0.029936
2	num__EXT_SOURCE_2	0.026068
3	cat__NAME_EDUCATION_TYPE_Higher education	0.014903
4	cat__NAME_INCOME_TYPE_Pensioner	0.014211
5	cat__CODE_GENDER_F	0.012579
6	cat__CODE_GENDER_M	0.011894
7	cat__NAME_EDUCATION_TYPE_Secondary / secondary...	0.010843
8	num__EXT_SOURCE_1	0.010649
9	cat__FLAG_OWN_CAR_Y	0.009597
10	num__CREDIT_GOODS_RATIO	0.009473

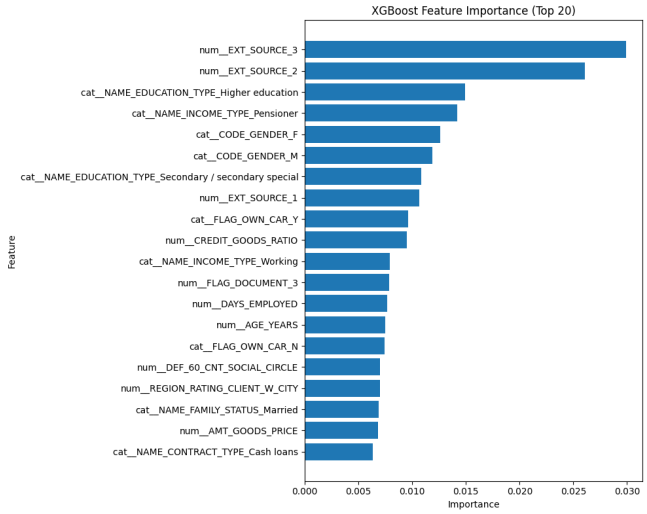


Figure 4: Feature importance of the final XGBoost model.

4. MLP Experiments

4.1 Architecture Comparison

I first compared three hidden-layer architectures: $(64,)$, $(128, 64)$, and $(256, 128, 64)$. Table 8 presents the validation results.

Table 8: MLP validation performance for different hidden-layer architectures

Model	Acc.	Prec.	Rec.	F1	AUC-PR	Thr.	Time	Iter.
MLP_arch_(64,)	0.8231	0.1935	0.3762	0.2556	0.1858	0.1545	129.26	100
MLP_arch_(128,64)	0.7954	0.1537	0.3405	0.2118	0.1414	0.0965	287.92	100
MLP_arch_(256,128,64)	0.8316	0.1515	0.2360	0.1845	0.1276	0.0307	550.15	100

The best architecture was **(64,)** with validation AUC-PR = 0.185776. This is a notable change from the earlier run: the shallowest network now performs best. The deeper models were substantially slower and failed to improve ranking quality. This suggests that, under the current preprocessing and optimization setup, larger MLPs were harder to optimize effectively and did not convert their extra capacity into better generalization.

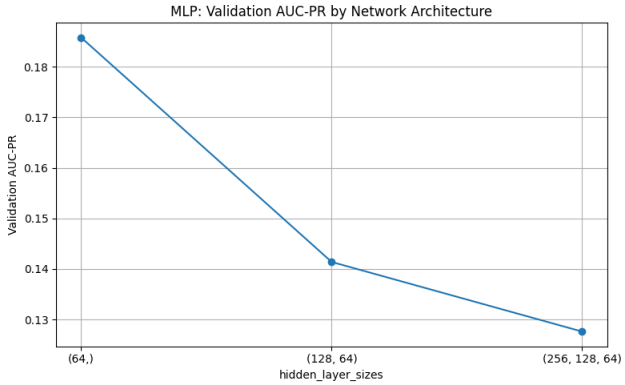


Figure 5: Effect of MLP architecture on validation AUC-PR.

4.2 MLP Hyperparameter Tuning

Using architecture $(64,)$, I tuned activation function, initial learning rate, and maximum number of iterations. The validation results are reported in Table 9.

Table 9: MLP validation performance under different parameter settings

Model	Acc.	Prec.	Rec.	F1	AUC-PR	Thr.	Act.	LR	Iter.	Time
MLP_tuned	0.8253	0.2159	0.4423	0.2902	0.2066	0.1423	relu	0.010	50	117.11
MLP_tuned	0.8241	0.2034	0.4041	0.2706	0.1961	0.1417	tanh	0.010	50	59.63
MLP_tuned	0.8228	0.1992	0.3958	0.2650	0.1955	0.1227	relu	0.001	50	66.00
MLP_tuned	0.7938	0.1809	0.4407	0.2565	0.1919	0.1061	tanh	0.001	50	61.22
MLP_tuned	0.8231	0.1935	0.3762	0.2556	0.1858	0.1545	relu	0.001	100	131.49
MLP_tuned	0.7799	0.1696	0.4431	0.2453	0.1717	0.1081	relu	0.001	150	197.43
MLP_tuned	0.0860	0.0810	0.9970	0.1498	0.0810	0.0735	relu	0.100	50	17.53

The best validation configuration was

$$\texttt{activation} = \texttt{relu}, \quad \texttt{learning_rate_init} = 0.01, \quad \texttt{max_iter} = 50$$

This setting achieved the best validation AUC-PR, **0.206608**, and the highest validation F1-score among the candidate MLP settings. The network was clearly sensitive to optimization choices. In particular, `learning_rate_init` = 0.1 led to a degenerate solution with recall near 1.0 but extremely poor precision and AUC-PR. This indicates unstable optimization and confirms that MLP is much more sensitive to hyperparameter choice than XGBoost. The selected validation threshold for the best MLP was **0.142259**.

4.3 Final MLP Test Performance

The final MLP was retrained on train + validation data and then evaluated on the test set. Results are shown in Table 10.

Table 10: Final MLP test performance

Model	Acc.	Prec.	Rec.	F1	AUC-PR	Thr
Final_MLP	0.8495	0.2390	0.3958	0.2980	0.2179	0.142

The final MLP improved substantially compared with the older baseline version. On the test set, it achieved recall 0.395811 and F1-score 0.298019, which are much more meaningful for this imbalanced problem. However, its AUC-PR remained below XGBoost, and its total training time was much longer.

Table 11: Final MLP classification report on the test set

Class	Precision	Recall	F1-score	Support
0	0.94	0.89	0.92	42,403
1	0.24	0.40	0.30	3,724
Accuracy		0.85		46,127
Macro Avg	0.59	0.64	0.61	46,127
Weighted Avg	0.89	0.85	0.87	46,127

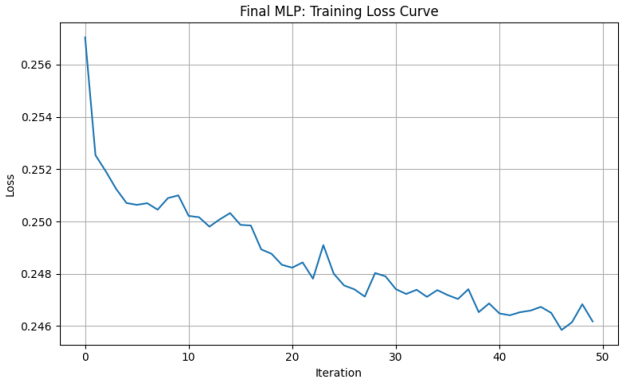


Figure 6: Training loss curve for the final MLP.

5. GBDT vs. MLP Comparison

The final side-by-side comparison is shown in Table 12.

Several conclusions follow from this final comparison.

First, **XGBoost achieved the best ranking performance**, with a test AUC-PR of 0.250945 compared with 0.217903 for MLP. Because the dataset is imbalanced, AUC-PR is one of the most important metrics in this study. On that measure, XGBoost is clearly stronger.

Second, **XGBoost also achieved the best F1-score and recall**. Its recall for the positive class was 0.457035, compared with 0.395811 for MLP, and its F1-score was 0.311579 versus 0.298019. This means that after threshold selection, XGBoost identified more default cases while maintaining comparable precision.

Third, **MLP achieved slightly higher accuracy**, 0.849459 versus 0.836950 for XGBoost. However, in an imbalanced dataset, accuracy is not the best indicator of model usefulness. The higher accuracy of MLP does not compensate for its lower AUC-PR and lower recall on the positive class.

Fourth, **XGBoost was also much faster in the final model training stage**. The final XGBoost training time was 65.07 seconds, whereas the final MLP required 223.88 seconds. Thus, in this updated experiment, XGBoost not only performed better on the key ranking and minority-class metrics, but also trained more efficiently.

5.1 When to Prefer Each Model

For this dataset, I would prefer **XGBoost**. The main reasons are:

- It achieved the highest AUC-PR and the strongest F1-score.
- It recovered more positive default cases than MLP.
- It provided direct feature importance for interpretation.
- It trained faster than the final MLP in this updated setup.
- Its behavior across hyperparameter settings was more stable.

I would consider **MLP** only if I had reasons to believe that a more expressive neural model, better feature representation, or a larger-scale tuning effort would unlock advantages not visible here. In the present tabular setting, that did not happen.

5.2 Interpretability

Interpretability differs substantially between the two models. XGBoost provides feature importance scores and therefore allows a direct inspection of which variables matter most for prediction. In this project, the top-ranked features included `EXT_SOURCE_1/2/3`, education variables, income type, gender, and financial ratio features. By contrast, MLP behaves much more like a black box. While its loss curve can be plotted, its hidden-layer parameters are not easily interpretable in business terms.

5.3 Categorical Features and Missing Values

Both models required explicit preprocessing for missing values and categorical features in my implementation. One-hot encoding worked for both models after imputation. Still, the results suggest that XGBoost handles this style of structured tabular data more effectively. MLP needed standardized numeric variables and careful threshold selection, and it remained more fragile under optimization changes.

5.4 Hyperparameter Sensitivity

The experiments indicate that **MLP was more sensitive to hyperparameters than XGBoost**. In particular, an excessively large learning rate caused near-collapse in MLP performance. XGBoost also responded to learning rate and subsampling, but the degradation was smoother and the model remained easier to tune.

6. Discussion and Limitations

The strongest result of this study is that XGBoost outperformed MLP on the most important metrics for this task. The updated threshold-based evaluation changed the absolute values dramatically compared with the earlier baseline: both models now achieve much higher recall and F1-score. Even after that improvement, XGBoost still maintains a clear lead in AUC-PR and F1, which strengthens the conclusion that boosted trees are better suited for this structured credit-risk dataset.

Table 12: Final comparison between XGBoost and MLP on the test set

Model	Acc.	Prec.	Rec.	F1	AUC-PR	Thr.	Time
Final_XGBoost	0.8370	0.2364	0.4570	0.3116	0.2509	0.6327	65.07
Final_MLP	0.8495	0.2390	0.3958	0.2980	0.2179	0.1423	223.88

A second important observation is that threshold selection matters a great deal in imbalanced classification. Under the default threshold, both models previously appeared overly conservative and achieved very low recall. After selecting thresholds on the validation set, both models became much more useful in identifying positive cases. This confirms that threshold tuning is an important part of model evaluation when the positive class is rare and the practical objective is not simple accuracy maximization.

There are also several limitations:

- The hyperparameter search space was still limited rather than exhaustive.
- The MLP architecture was relatively simple and based on scikit-learn rather than a more flexible deep learning framework.
- Categorical variables were one-hot encoded instead of represented through embeddings.
- The evaluation focused on standard classification metrics rather than explicit business cost functions.

Despite these limitations, the comparison is controlled and meaningful because both models were trained on the same dataset and evaluated on the same train/validation/test split.

7. Conclusion

This report compared XGBoost and MLP on the Home Credit Default Risk dataset using the same dataset split and a consistent model-selection procedure. The updated results show that XGBoost is still the stronger model for this problem. It achieved the best validation AUC-PR, the best final test AUC-PR, the strongest F1-score, and the highest positive-class recall. It was also faster than the final MLP in this updated run. MLP improved substantially once threshold tuning was introduced, but it remained inferior on the key metrics that matter most for imbalanced credit-default prediction.

In summary, for this structured, mixed-type, imbalanced tabular dataset, **XGBoost is the preferred model**. The results reinforce the general lesson that boosted trees remain extremely competitive for real-world tabular classification tasks, especially when interpretability, ranking quality, and training efficiency are all important.

AI Tool Disclosure

How AI helped: I used ChatGPT mainly as a coding and writing assistant. It helped me troubleshoot minor Python and Scikit-learn bugs, catch syntax errors, and smooth out the phrasing and formatting of my README. My personal contributions: All the core modeling and analysis are entirely my own work. I independently designed the ML pipeline, strictly enforced the data leakage prevention steps, chose which hyperparameters to tune, and set up the evaluation logic. I also wrote and finalized the PDF report myself, including the data interpretation, model comparisons, and insights drawn from the loss curves and feature importance charts.